

L1: Project Presentation + Agile Methodology

Shahid Manzoor

Fall 2026

Project Presentation

Your Mission, Should You Accept It

(we suggest you do)

Group Objective

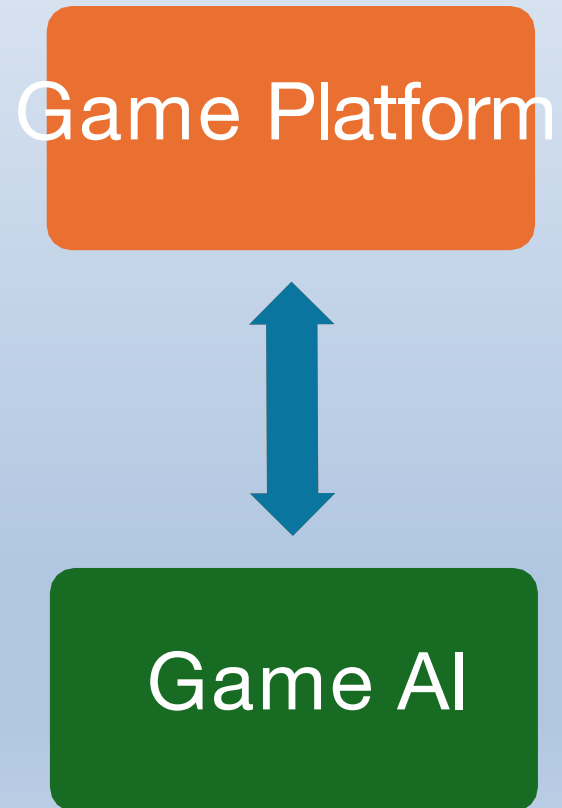
- Finish the implementation of a small-sized game

Learning Objectives

- Collaboration + Cooperation + Teamwork
- Requirements engineering
- Design + Development + Testing + Integration
- Scrum as agile methodology
- ...

The Mission 1

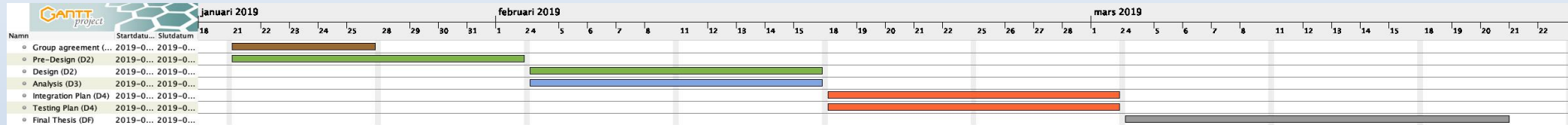
Implement a game with two main components



The Mission 2

- Your group will design, implement and test one component.
- Your group will be able to obtain the other component from another group.
- Your group must integrate that component with your own.
- While it is almost certainly more work and riskier, your group could choose to implement also the other component yourselves.

Timeline Overview: Three Sprints



Sprint 1 (wk. 38)

- Group agreement
- Requirements engineering
- Design
- Start of implementation

Sprint 2 (wks. 39 – 41)

- Specification
- Implementation

Sprint 3 (wks. 42 – 44)

- Integration
- Risk analysis
- Testing plan

Please be aware that details could change as the course progresses, due to unforeseen circumstances.

Week 38 (Sprint 1)

Objectives

- Requirements Elicitation for your component
- Scrum Coach meeting P1, Friday wk. 38 (2026-00-00)

Deliverables

- Kanban board URL K1, Tuesday wk. 38 (2026-00-00)
- Group agreement
- Requirements elicitation D1, Thursday wk. 38 (2026-00-00)
- Artefacts D2, Thursday wk. 39 (2026-00-00)

In addition

- Ideally, begin design and implementation A1, Friday wk. 39 (2026-00-00)

Week 39 (Sprint 2)

Objectives

- Design your component (ideally finish the design)
- Implement your component
- Optionally meet the client to show a prototype

Deliverables

- Kanban backlog exercise
- K2, Monday wk. 40 (2026-00-00)

Week 40 (Sprint 2)

Objectives

- Finish the design of your component
- Finish the implementation of your component

Deliverables

- Requirements specification D3, Sunday wk. 40
(2024-00-00)

Week 41 (Sprint 2/3)

Objectives

- Scrum Coach meeting P2, Monday wk. 41 (2026-00-00)
- Meet client (if needed book time)

Deliverables

- Kanban board screenshot K3, Monday wk. 41 (2026-00-00)
A2, Monday wk. 41 (2026-00-00)
- Artefacts

Week 42 (Sprint 3)

Objectives

- Development work
- Plan integration work

Deliverables

- Risk assessment plan
- Integration plan

D4, Sunday wk. 42 (2026-00-00)

D5, Sunday wk. 43 (2026-00-00)

Week 44 (Sprint 3)

Objectives

- Scrum Coach meeting P3, Monday wk. 44 (2026-00-00)
- Integration work
- Create testing plan

Deliverables

- Kanban board screenshot K4, Monday wk. 44 (2026-00-00)
Artefacts A3, Monday wk. 44 (2026-00-00)

Week 45 (Sprint 3)

Objectives

- Finish integration work

Deliverables

- | | |
|----------------------|--------------------------------|
| • Testing Plan | D6, Monday wk. 45 (2026-00-00) |
| • Project Completion | DF, Sunday wk. 46 (2026-00-00) |

Opportunity to Resubmit Deliverables

- If you have submitted a deliverable D_x on time...
- ... you are allowed to resubmit it later.
- It will be assessed along with the next deliverable D_y ($y > x$).
- This means you can react to feedback given to a deliverable and improve it.
- Note that while we do not recommend handing in an incomplete deliverable, it is better than not handing in anything at all.

Integration Process

Collecting components

- We will collect information about the components being implemented by each group by Sunday wk. 6 (2026-00-00).

Getting the other component

- Your team will get the collected information about available components and should make a choice about which one to use.

Agile Methodology

Software Development Methodology

We will discuss why software development has been notoriously fraught with delays and with failed projects.

We will discuss the evolution of software development methodology from waterfall towards agile methodology.

We will discuss some particular flavours of agile methodology, in particular Scrum, Kanban and (to a lesser extent) Extreme Programming (XP)

How Hard Can It Be?

- Consider building construction, in particular skyscrapers.
- They are very complex projects, involving hundreds of people and dozens of contractors, yet they usually finish on time and on budget.
- Then consider software development projects.
- History is rich in projects that
 - Took significantly longer than planned (often twice or three times)
 - Were abandoned or resulted in software that was never used
 - Cost significantly more than expected
 - Were obsolete by the time they were delivered

Why is building construction predictable?

- Buildings are almost always built using technology and plans that is very similar to those for previous buildings.
- There are few surprises, so you can make a reliable plan and stick with it. Why is that?
 - Components and materials are almost always standard “off the shelf”.
 - Contractors and vendors can reliably estimate the number of workers needed and the time needed for each subproject.
 - There are heavy penalties for contractors and vendors if not delivering on time, so they are heavily incentivized to deliver on time.
 - Detail: construction companies can hire police to control traffic

Why is software construction so unpredictable?

There are many reasons, for example:

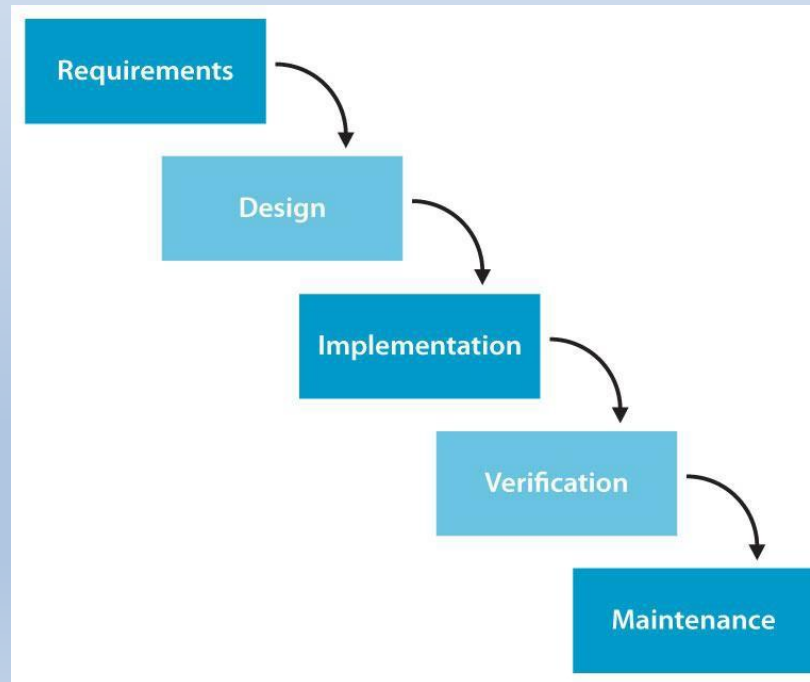
- The customer often has an incomplete idea of what they or their end users want.
- Software developers are notoriously poor at estimating the time to develop anything beyond about two days.
- Specifications are often ambiguous, leading to different interpretations between producers and consumers of an interface.
- Developers make mistakes that may surface only much later.

Building large systems

- The military has known for a long time that groups with more than about seven people (or seven subgroups) cooperate poorly on complex tasks.
- This applies also to software development.
- We may try breaking down a large software development project into components, subcomponents, etc., until we get to units that can be built by small teams.
- The resulting complexity of such a plan is virtually guaranteed to make it fail or run into difficulties.
- We need a better methodology.

The Waterfall model

- This model builds on the (in hindsight) unrealistic expectation that we know from the start exactly what should be delivered from a large project.
- Here is a visual:



The Waterfall Model

In its purest form, we follow five subsequent steps:

1. Gather all requirements from the customer
2. Design the product and its architecture
3. Implement all components
4. Verify that all components work according to the design
5. Maintain the product for its lifetime

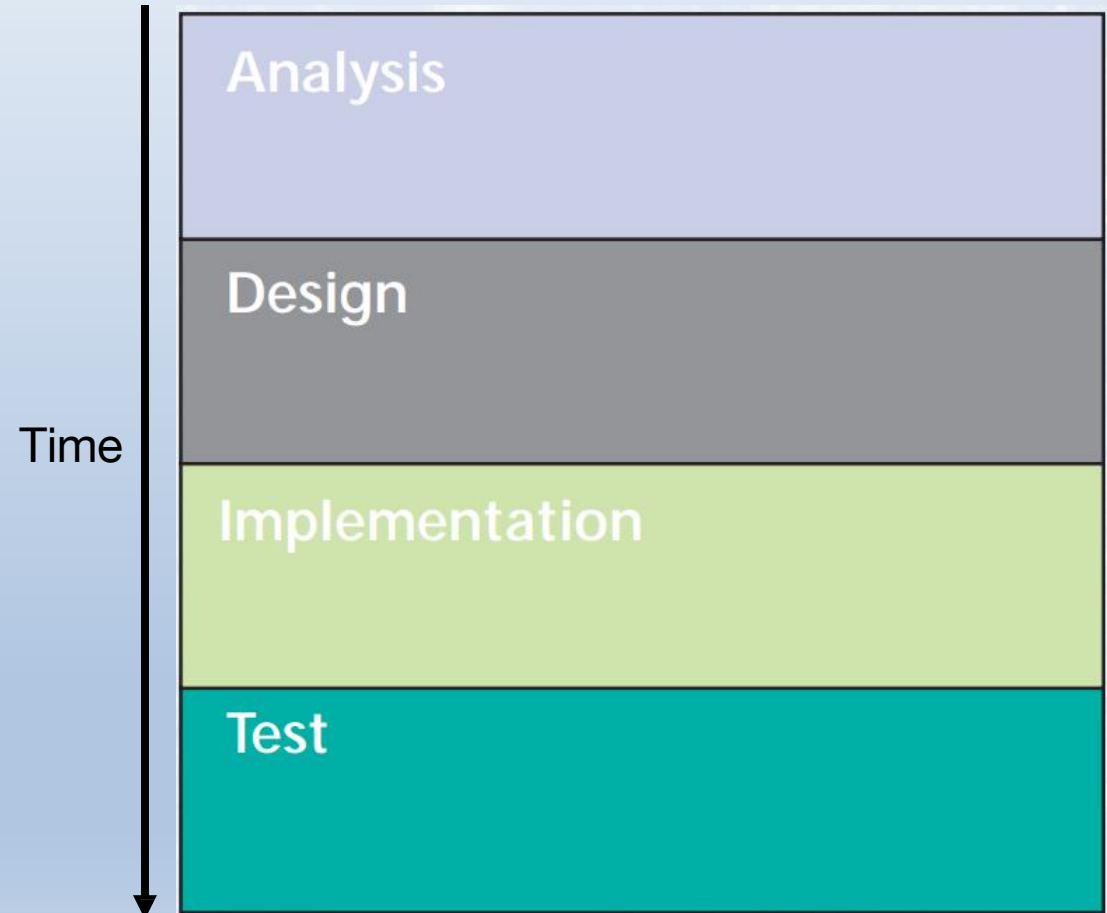
This works well for a very small piece of software. It proved disastrous as a methodology for large systems.

Aside: When a Project Falls Behind

- If a large software project falls behind schedule, one temptation may be to add more people to the project, unfortunately often making things worse.
- In the oldie-but-goldie book [The Mythical Man-Month](#) (originally published in 1975 with an anniversary edition in 1995) Fred Brooks explains why this is the case.
- The book can be seen as a philosophical tenet of agile development.

The Waterfall Model Visually

- The exact steps differ, e.g., they could be
 - Analysis
 - Design
 - Implementation
 - Test
- Here's another way to visualize development following the waterfall model.

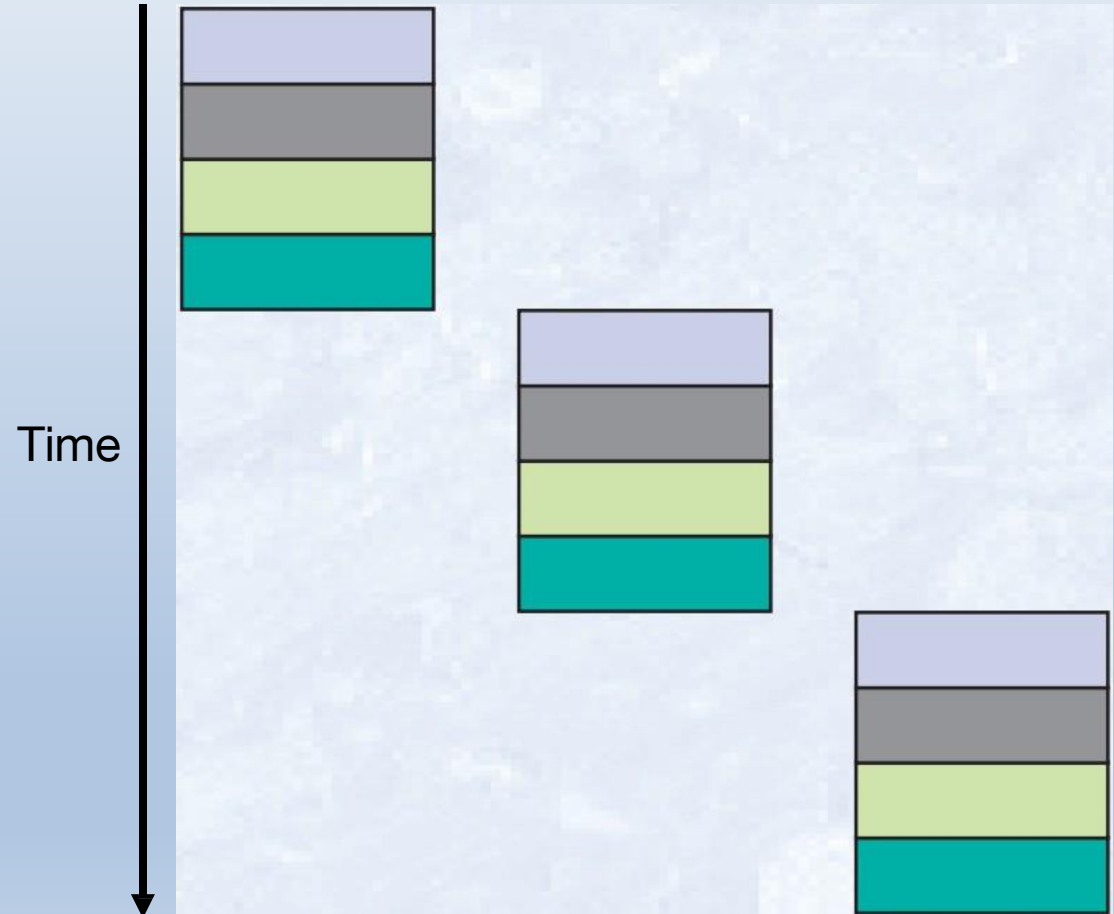


Iterative Approaches

- As a way of tackling one of the fundamental problems with waterfall, we may consider not designing and implementing the whole product in one giant step.
- Suppose we could identify some core functionalities that would constitute a *minimal viable product* (MVP)?
- Then we could build this MVP using similar steps as in waterfall, but for a much more manageable project.
- Then we gather new requirements for a minimal extension of the MVP to get an MVP₂, and so we keep iterating.

An Iterative Approach

- We can visualize an iterative approach like this.
- It's like mini-waterfalls, but each one is more
- Also, requirements can be revisited and reprioritized between iterations.



Aside: The Manifesto for Agile Software Development (2001)

Based on twelve principles (my highlights):

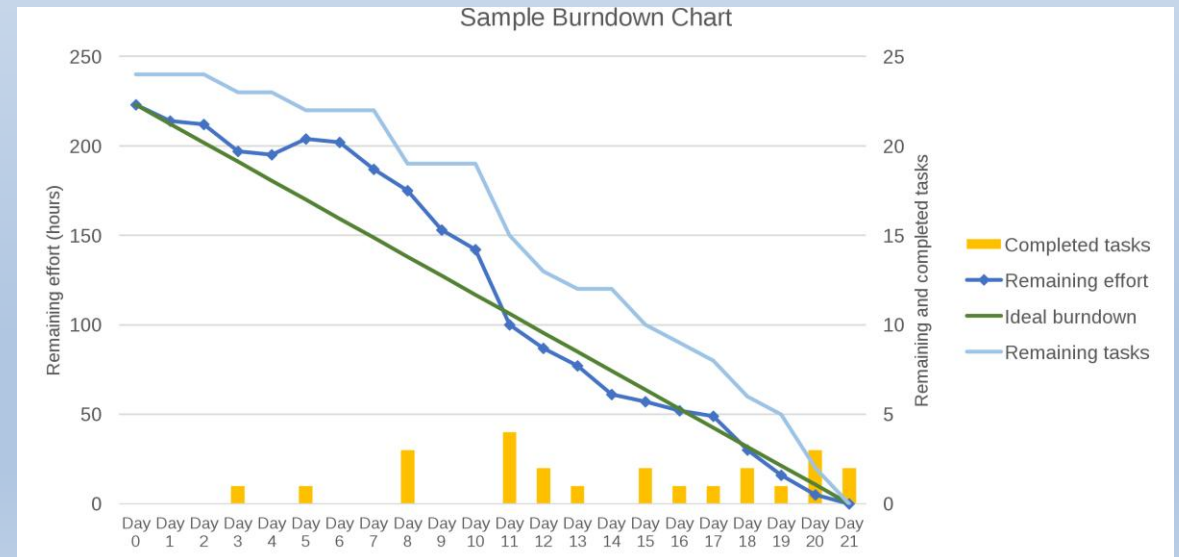
1. Customer satisfaction by **early and continuous** delivery of valuable software.
2. Welcome changing requirements, even in late development.
3. **Deliver working software frequently** (weeks rather than months).
4. Close, daily cooperation between business people and developers.
5. Projects are built around motivated individuals, who should be trusted.
6. Face-to-face conversation is the best form of communication (**co-location**).
7. Working software is the primary measure of progress.
8. Sustainable development, able to maintain a constant pace.
9. Continuous attention to technical excellence and good design.
10. Simplicity—the art of **maximizing the amount of work not done**—is essential.
11. Best architectures, requirements, and designs emerge from self-organizing teams.
12. Regularly, the team **reflects on how to become more effective**, and adjusts accordingly.

Scrum Methodology

- Work in iterations (sprints) of 1 to 4 weeks.
- Each sprint:
 - Starts with a sprint planning event, choosing work items from a backlog.
 - Daily short stand-up meetings (“scrums”) for brief progress reports and blocking issues (to be resolved after the scrum).
 - A “scrum master” educates the team and manages the process.
 - At the end, carry out a sprint review (what was done, or not done) and a sprint retrospective (to improve processes).
 - The sprint can be aborted early.
- Ideally a sprint produces an increment, an output that could be released.

Scrum methodology

- Maintaining the product backlog and prioritizing work items is crucial.
- During a sprint the selected work items are kept in a sprint backlog to track the work.
- A “burndown chart” is often used to visualize progress in the form of work done and (estimated) work remaining.



Kanban

- Kanban has its roots in lean manufacturing processes (such as for cars) in Japan (cf. Taiichi Ohno working for Toyota).
- The name can be seen as referring to a signboard that is an essential part of the methodology.
- Kanban an origin for notions such as just-in-time processing.
- A key idea is to limit “work in progress” to avoid excessive build-up of intermediate parts that won’t get used for a long time.
- The time spent on producing such parts should instead be spent on a step that is falling behind.

Kanban core principles

This version is from *Getting Started with Kanban for Software Development* by David J. Anderson and Janice Linden-Reed:

Principle	Details
1. Visualize the Workflow	Represent the work items and the workflow on a card wall or electronic board.
2. Limit Work-in-Progress (WIP)	Set agreed upon limits to how many work items are in progress at a time.
3. Measure & Manage Flow	Track work items to see if they are proceeding at a steady, even pace.
4. Make Process Policies Explicit	Agree upon and post policies about how work will be handled.
5. Use Models to Evaluate Improvement Opportunities	Adapt the process using ideas from Systems Thinking, W. E. Deming, etc.

1. Visualize the Workflow

- A traditional Kanban board is a physical board with movable cards typically on a wall near the team's work places.
- The board has one or two columns for each activity (such as Analysis, Development, Testing). Together the activities form a “value creation network.”
- Each card identifies a work item (e.g., a feature, a story or a task) and contains information such as a title, a reference id and the date it entered the system.
- As a work item progresses through the network, its card moves.
- The wall should represent the current development process with an intent to improve it over time.

2. Limit Work-in-Progress (WIP)

- Each column has a limit on the number of work items it may contain at any given time. This is arguably the “secret sauce” of Kanban.
- A card cannot move from one column to another if that would violate the WIP limit of the new column. The work item is blocked.
- If this seems counterproductive, consider what would happen if there were no limit. Work would pile up, getting delayed.
- When work is blocked due to a WIP limit, the team should address it by moving resources and possibly adjusting other WIP limits.
- This happens naturally as the blockage tends to affect earlier columns, and some team members will be freed

3. Measure and Manage Flow

- The ideal, stable situation when using Kanban is that work items flow through the network at a steady pace (to be measured).
- When work items sit blocked, obstructing the steady flow, a conversation needs to happen about how to resolve it. Some possibilities:
 - Team members could “swarm” to work items that are taking longer than expected to complete and are the source of the blockage.
 - WIP limits could be increased slightly to the right of the blockage to allow some work items to move on.
 - WIP limits could be decreased slightly to the left of the blockage to slow down inflow to the congested part.

4. Make Process Policies Explicit

- The team agrees on principles for how to manage flow.
- The team can agree to change these principles as needed.
- Focusing on maintaining the principles minimizes contentious discussions about individual work items, reducing conflict.
- At least some of the policies, e.g., the current WIP limits, should ideally be posted on the board.

5. Use Models to Evaluate Improvement Opportunities

- Scientific work has been done on how to optimize work flow when using systems such as Kanban.
- Teams should use such methods or at least periodically review data on performance such as flow time, WIP, and delivery throughput, using it to make adjustments to policies.

Online Kanban Boards

- It is generally agreed that having a physical Kanban board and a team working in physical proximity to each other and the board is the most productive environment.
- However, an online Kanban board allows for a geographically distributed team, which is often unavoidable.
- There are various services providing online Kanban boards and tools to go with them, one being [Trello](#).
- For work tracking in your project, and for satisfying certain course deliverables, we suggest using a free Kanban board from Trello, but you could agree to use something else.

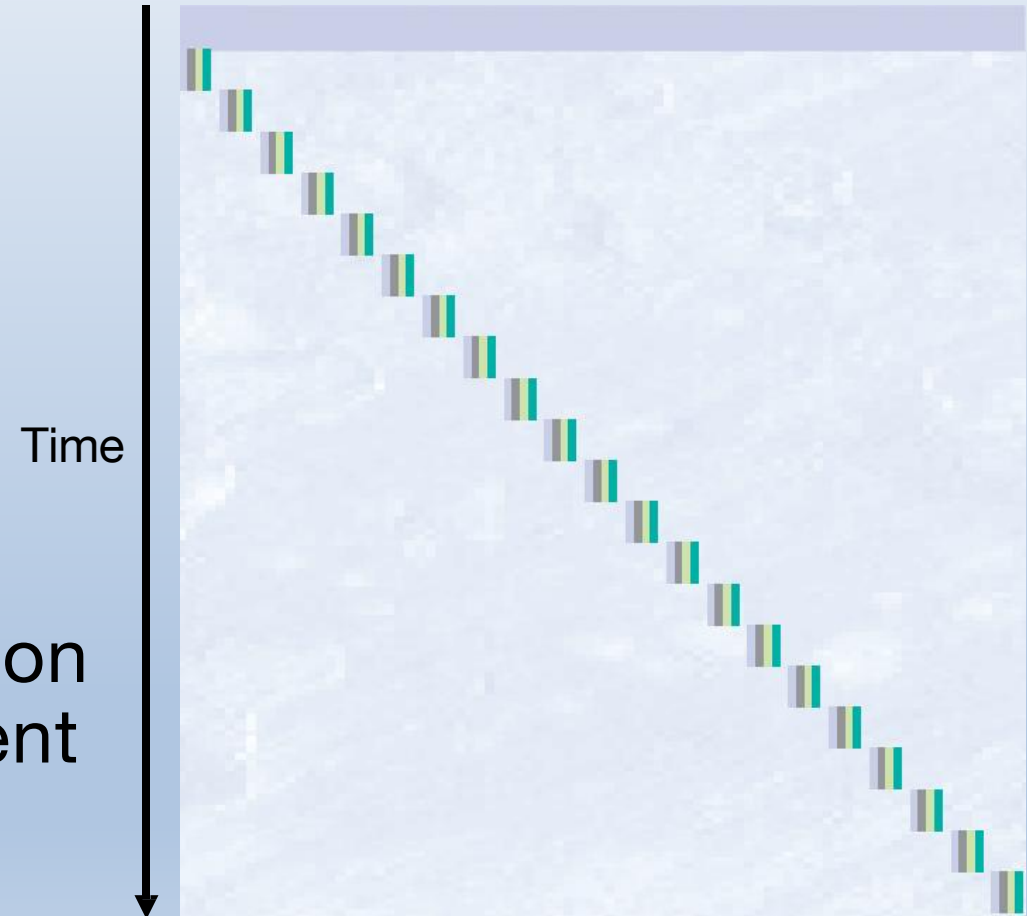
Learning More about Kanban

There are plenty of resources, such as:

- *Getting Started with Kanban for Software Development* by David J. Anderson and Janice Linden-Reed
- *Agile Project Management with Kanban* by Eric Brechner, Microsoft Press, 2015.
- *Kanban: Successful Evolutionary Change for Your Technology Business* by David J. Anderson, Blue Hole Press, 2010.

Extreme Programming (XP)

- Another evolution of agile methodology calls itself Extreme Programming.
- We can visualize XP like this, like very short iterations where in each iteration, work items on analysis, design, development and test are carried out in parallel.



XP Practices

Here are some of the Extreme Programming recommended practices:

- Planning Game: The customer decides scope and timing of releases (MVP) based on estimates from team. Team implements only what is needed by the stories in the current iteration.
- Small Releases: First version in production within months, then frequent (weekly or daily) releases.
- Simple Design: Minimize duplicate code, fewest possible classes and methods.
- Tests: Developers write unit tests for every piece of code. Customers write functional tests. All tests should always pass.

XP Practices

- Refactoring: Evolve the product by evolution of the original design while keeping all tests running.
- Pair Programming: All production code is written by two people sharing a workstation.
- Continuous Integration: New code is integrated with the current system within hours. (The result is built from scratch and unless all tests pass, the integration is rejected.)
- Collective ownership: Anyone is allowed to improve any code.
- On-site customer: A customer sits with the team full time.

XP Practices

- 40-hour weeks: Successive weeks of overtime is forbidden. Need for overtime is a sign of deeper problems.
- Open Workspace: No individual offices, everyone has a desk on the perimeter of a shared workspace. Pair programmers work on workstations set up in the center.
- Rules Can Change: Everyone must follow the rules. But everyone is encouraged to propose improvements to the rules and the team can change the rules at any time.

Learn More about Extreme Programming

- There are plenty of books on the subject, e.g.,
 - *Extreme Programming Explained: Embrace Change*, 2nd Ed. by K. Beck & C. Andres, Addison-Wesley, 2004.
- You could start with the article [Embracing Change with Extreme Programming](#) by K. Beck.

What Agile Methodology Is Best?

- Data suggests any one of Scrum, Kanban and XP outperforms waterfall development for large projects.
- Many teams and companies pick some combination of principles and methods from the various agile methodologies.
- Use this course to try them out as much as you can.
- Note that all methodologies emphasize that the team must come to agreement about a set of rules applying to everyone.

Trello Demo

- I will futz around with a Trello board, simulating the beginning of a project.
- <https://trello.com>
- A free account has limited abilities, but enough to be useful for a small project.